3D Sail Tracking with Twist

Model Overview

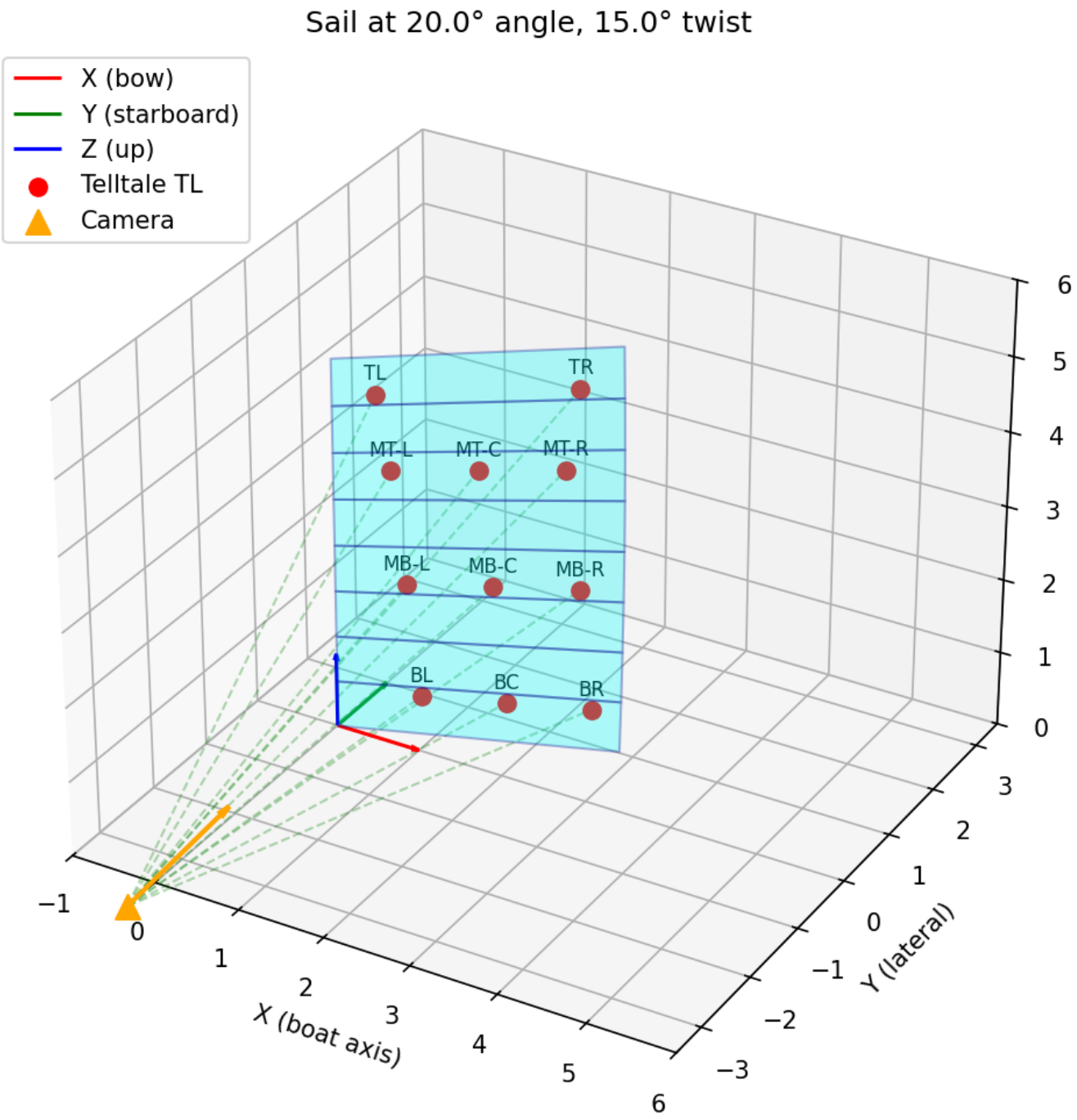
The sail is modeled as a **ruled surface** in 3D space with two degrees of freedom:

Parameter	Description	Range
Base Angle (α)	Rotation of sail at the foot around the mast (Z-axis)	0° – 45°
Twist (τ)	Additional rotation at the head relative to the foot	0° – 30°

The effective rotation angle varies linearly along the sail height:

```
angle(v) = α + v × τ
```

Where `v ∈ [0, 1]` is the normalized height (0 = foot, 1 = head).



Input Parameters

Sail Geometry (`SailGeometry`)

Field	Type	Description
`width`	float	Sail width in meters (luff to leech)
`height`	float	Sail height in meters (foot to head)
`mast_position`	(x, y, z)	Mast base position in world coordinates
`telltales`	list	Telltale positions as normalized (u, v) coordinates

Camera Configuration (`CameraConfig`)

Field	Type	Description
`position`	(x, y, z)	Camera position in world coordinates
`look_at`	(x, y, z)	Target point camera is facing
`focal_length`	float	Focal length in pixels
`principal_point`	(cx, cy)	Image center in pixels
`image_size`	(w, h)	Image dimensions in pixels

Tracker Parameters

Field	Default	Description
`alpha`	0.7	Weight for distance cost
`beta`	0.3	Weight for confidence cost
`max_distance`	0.3	Max normalized distance for valid match
`coarse_steps`	7	Angle grid resolution
`coarse_steps_twist`	5	Twist grid resolution

Output Values

The tracker returns a tuple `(tracks, base_angle, twist)`:

Output	Type	Description
`tracks`	`list[Track]`	Matched detections with telltale IDs
`base_angle`	float	Estimated sail rotation at foot (degrees)
`twist`	float	Estimated twist angle (degrees)

Each `Track` contains the original `Detection` plus assigned `track_id` (telltale ID).

Optimization Algorithm

1. Coarse 2D Grid Search

Evaluates cost on a discrete grid of (angle, twist) combinations:

- Grid size: `(coarse_steps + 1) × (coarse_steps_twist + 1)` = 48 evaluations
- At each grid point: project telltales → build cost matrix → Hungarian assignment

2. Continuous 2D Refinement

Starting from the best grid point, uses **L-BFGS-B** optimizer:

- Bounded within ±5° of coarse solution

- Max 20 iterations, tolerance 1e-4
- Converges to local minimum

3. Final Assignment

At refined (angle, twist), performs Hungarian assignment and filters matches by `max_distance`.

Cost Function

```
cost(i, j) = α × d_normalized(detection_i, telltale_j) + β × (1 −
confidence_i)
```

Where `d_normalized = euclidean_distance / image_diagonal`

Error Characteristics

Condition	Angle Error	Twist Error
No noise	< 4°	< 6°
Light noise (10px σ)	< 5°	< 7°
Heavy noise (20px σ)	< 8°	< 10°

Note: Angle and twist can exhibit some coupling — similar 2D projections can arise from different (α, τ) pairs, especially when twist is near zero.

Performance

Metric	Value
Average latency	~16 ms
Throughput	~60 FPS
Grid evaluations	48
Refinement iterations	~10-20

The system supports **real-time tracking** at 30+ FPS on standard hardware.

Usage Example

```
from models.sail_3d import Sail3DConfig
from sail_3d_tracker_v2 import Sail3DTrackerV2

# Load configuration
```

```
config =
Sail3DConfig.from_json_file("assets/configs/sail_3d_example.json")

# Create tracker
tracker = Sail3DTrackerV2(config)

# Process detections
tracks, angle, twist = tracker.update(detections)

print(f"Sail angle: {angle:.1f}°, Twist: {twist:.1f}°")
print(f"Matched {len(tracks)}/{len(config.sail.telltales)} telltales")
```

File Structure

```
src/
├── models/sail_3d.py        # Data models (SailGeometry, CameraConfig,
Sail3DConfig)
├── projection.py            # 3D→2D projection with twist
├── sail_3d_tracker.py       # 1-DOF tracker (angle only)
├── sail_3d_tracker_v2.py    # 2-DOF tracker (angle + twist)
└── visualization_3d.py      # Debug plotting utilities
```